Computational Complexity, DP, and recursion
Quiz 2 Practice

10-607

Disclaimer: This isn't exactly a "specimen paper." That is, some problems here are going to be more difficult, and some are going to be easier. However, this is a good place to start to find out which concepts you need to work on more. This also isn't meant to reflect the length or format of the actual quiz.

1. Let $A = a_1, a_2, ..., a_N$, $B = b_1, b_2, ..., b_N$, and $C = c_1, c_2, ..., c_N$ be three sequences of length N. The longest common subsequence is the longest sequence $S = s_1, ..., s_K$ such that S is a subsequence of A, B, and C. Note that a subsequence is defined as any sequence that can be formed by deleting one or more characters from a sequence. For example, "abd" and "be" are valid subsequences of the sequence "abcde". Provide a dynamic programming algorithm that, given sequences A, B, and C, finds the length of their longest common subsequence. Briefly explain why your algorithm is correct and provide the runtime.

   Ans:

   Initialize $Q(i, j, k) = 0$ if $i = 0$ or $j = 0$ or $k = 0$. This is because if at least one string is empty, there cannot be a common subsequence. The recurrence relation for dynamic programming is:

   $Q(i, j, k) = Q(i - 1, j - 1, k - 1) + 1$ (if $a_i = b_j = c_k$)

   $Q(i, j, k) = \max\{Q(i - 1, j, k), Q(i, j - 1, k), Q(i, j, k - 1)\}$ (if $a_i \neq b_j$ or $b_j \neq c_k$ or $c_k \neq a_i$)

   Runtime for this algorithm is $O(N^3)$.

2. Rank the following functions in order of their asymptotic growth. That is if $f_i(n) = O(f_j(n))$ then $f_i(n)$ should come before $f_j(n)$ in your list. If $f_i(n) = \Theta(f_j(n))$ then the two functions should be given the same rank.
   - $f_1(n) = n^5$
   - $f_2(n) = nlog_2n$
   - $f_3(n) = 10^{10}$
   - $f_4(n) = 2^{log_2n}$
   - $f_5(n) = 2nlog_{10}n$

- $f_6(n) = (n+1)!$
- $f_7(n) = 2log_{10}n$
- $f_8(n) = n^{log_2 n}$
- $f_9(n) = n(log_2 n)^3$

Ans: To get the most out of this problem, formally prove everything.

3

7

4

2, 5

9

1

8

6

3. Prove or disprove: Let $f(n)$ and $g(n)$ be functions that take on positive values for all $n > 1$ and such that $f(n) = O(g(n))$. Then $2^{f(n)} = O(2^{g(n)})$

Ans: False. Come up with a valid counter-example. Consider: $f(n) = 2n$, $g(n) = n$. We leave it as an exercise to the reader to show why this is not true.

4. Consider the following algorithm:

Algorithm 1 InsertionSort(A)

```
1: n ← length(A)
2: for i = 2 to n do
3:     x ← A[i]
4:     j ← i − 1
5:     while j ≥ 1  and  A[j] > x do
6:         A[j + 1] ← A[j]
7:         j ← j − 1
8:     end while
9:     A[j + 1] ← x
10: end for
```

Does this correctly sort an array A? What's the best case running time? What's the worst case running time?

Ans: Yes, this correctly sorts any given array. To see this, note that when the algorithm completes iteration i of the outer loop, the first i elements of A are in sorted order.

In the best case the array A is already sorted, in which case the conditional of the while loop always evaluates to false, and $O(1)$ work is done per iteration of the outer loop. Hence the best case runtime is $O(n)$. (In fact it is $\Theta(n)$).

In the worst case, the while loop iterates $\Omega(i)$ times for each i in the outer loop, which gives a runtime of $\sum_{i=1}^{n} \Omega(i) = \Omega(n^2)$. (In fact it is $\Theta(n^2)$).